

Plan
Enregistrement
Fichier Séquentiel
Récursivité
Liste
Pile
File
Arbre

COURS ASDII

Enseignantes:
JRIDI Wafa

1ère BI

ENREGISTREMENT

I. Définition:

Un type enregistrement permet de regrouper dans une même structure de données plusieurs informations de types différents. On parle donc d'un type compose et structure

Exemple:

Etant donné un étudiant qui est caractérisé par son nom, prénom, date de naissance et son numéro de carte d'étudiants.

Si nous voulons représenter dans une même variable ces informations, l'utilisation des enregistrements parait comme étant la solution la plus adaptée.

ENREGISTREMENT

II. Syntaxe de déclaration d'un type enregistrement:

Algorithme Nom_algorithme

déclaration de type → type structure Nom_enregistrement

Nom-champ1:type

Nom-champ2:type

:

:

fin structure Nom-Enregistrement

déclaration des variables et constantes

Debut

:

:

:

fin

ENREGISTREMENT

II. Syntaxe de déclaration d'un type enregistrement:

Exemple: structure Etudiant
 Nom: chaine de caractères
 Prenom: chaine de caractères
 Age: entier
 NCE: entier
 fin structure Etudiant

III. Syntaxe de déclaration d'une variable de type enregistrement:

Var Nom-varE: Nom-enregistrement

Exemple:

```
Algorithme Test
Type structure Etudiant
  Nom: chaine de caractères
  Prenom: chaine de caractères
  Age: entier
  NCE: entier
fin structure Etudiant
Var E:Etudiant
Debut
:
fin
```

} Déclaration d'un type
enregistrement de type etudiant

} Déclaration d'une variable de
type Etudiant

ENREGISTREMENT

IV. Accès aux champs d'un enregistrement:

Nom-VarE.nomchamp

Exemple:

E:Etudiant

E.Nom → Accès au champ Nom de la structure Etudiant

E.NCE → Accès au champ NCE de la structure Etudiant

V. Applications:

Exercice1:

Ecrire l'algorithme permettant de mémoriser les informations suivantes: Nom, prénom, adresse pour chaque personne travaillant dans un établissement.

ENREGISTREMENT

Solution:

Algorithme Carnet_Adresse

Type structure Personne

Nom: chaine de caractères

Prenom: chaine de caractères

Adresse: chaine de caractères

fin structure Personne

Var

i,n:Entier

P:personne

Tp:Tableau [1,,100] de Personne

Debut

Repeter

 ecrire ("donner le nombre de personnes:");

 lire(n);

Jusqu'à (n>=1) et (n < 100)

Pour i de 1 a n faire

 Ecrire («donner le nom: »); lire (TP[i].nom);

 Ecrire («donner le prenom: »); lire (TP[i].prenom);

 Ecrire («donner l'adresse: »); lire (TP[i].adresse);

Fin pour

ENREGISTREMENT

Exercice 2:

Modifier l'algorithme précédent si l'on considère que l'adresse est composée d'un numéro, d'une rue et d'un code postal.

Solution:

Algorithme Carnet_Adresse version2

Type

structure Adresse

Numéro: entier

Rue: chaine de caractères

Code postal: entier

fin structure adresse

structure Personne

Nom: chaine de caractères

Prenom: chaine de caractères

Adr: Adresse

fin structure Personne

Var

i,n:Entier

TP: Tableau [1..50] de Personne

ENREGISTREMENT

Debut

Repeter

ecrire ("donner le nombre de personnes:");
 lire(n);

Jusqu'à (n>=1) et (n<=50)

Pour i de 1 a n faire

Ecrire («donner le nom: »); *lire* (TP[i].nom);
 Ecrire («donner le prenom: »); *lire* (TP[i].prenom);
 Ecrire («donner Le numéro d'adresse: »); *lire* (TP[i].adr.numéro);
 Ecrire («donner la rue: »); *lire* (TP[i].adr.rue);
 Ecrire («donner le code postal: »); *lire* (TP[i].adr.codepostal);

Fin pour

fin

Exercice 3:

Ecrire l'algorithme permettant de classer les personnes par région (ville)

ENREGISTREMENT

Solution:

Algorithme Carnet_Adresse version3

Type

structure Adresse

Numéro: entier

Rue: chaine de caractères

Code postal: entier

fin structure adresse

structure Personne

Nom: chaine de caractères

Prenom: chaine de caractères

Adr: Adresse

fin structure Personne

Var

i,j,n:Entier

Tableau TP[50] :Personne

p:personne

Debut

Repeter

 ecrire ("donner le nombre de personnes:"); lire(n);

Jusqu'à (n>=1) et (n<=50)

ENREGISTREMENT

Pour i de 1 a n faire

Ecrire («donner le nom: »); lire (TP[i].nom);

Ecrire («donner le prenom: »); lire (TP[i].prenom);

Ecrire («donner Le numéro d'adresse: »); lire (TP[i].adr.numéro);

Ecrire («donner la rue: »); lire (TP[i].adr.rue);

Ecrire («donner le code postal: »); lire (TP[i].adr.codepostal);

Fin pour

Pour i de 1 a n faire

Pour j de i+1 a n faire

si (TP[j].adr.codepostal < TP[i].adr.codepostal)

alors

P ← TP[j];

TP[j] ← TP[i];

TP[i] ← P;

finsi

Fin pour

Fin pour

fin

ENREGISTREMENT

Exercice 4:

Etant donné un client qui est représenté par son code, son nom et son chiffre d'affaire, soit à gérer n clients, écrire l'algorithme qui permet de saisir les informations relatives aux différents clients et qui permet de déterminer:

- le client ayant le chiffre d'affaire le plus élevé
- le client ayant le chiffre d'affaire le plus faible
- la moyenne des chiffres d'affaires de tous les clients.

Solution:

Algorithme chiffre d'affaire

Type

structure Client

Nom: chaîne de caractères

Code: entier

CA: réel

fin structure client

Var

Trouve: booléen

Indmin,i,j,n,indmax:entier

S,min,max,moy:réel

Tableau T[50]:client

ENREGISTREMENT

debut

Repeter

 ecrire ("donner le nombre de clients:"); lire(n);

Jusqu'a (n>=1) et (n<=50)

 Ecrire («donner le nom: »); lire (T[1].nom);

 Ecrire («donner le code: »); lire (T[1].code);

 Ecrire («donner Le chiffre d'affaire: »); lire (T[1].CA);

Pour i de 2 a n faire

Repeter

 trouve←faux;

 ecrire ("donner le code:"); lire (T[i].code);

 j←1;

tant que (j<=i-1) et (non trouve) faire

 si (T[i].code)= (T[j].code)

 alors trouve←vrai;

 finsi

 j←j+1;

fintantque

Jusqu'a (trouve=faux)

 ecrire("donner le nom"); lire (T[i].nom);

 Ecrire («donner le chiffre d'affaire: »); lire (T[i].CA);

finpour

Max← T[1].CA; indmin ← 1; Min← T[1].CA; ; indmax ← 1; S ← 0;

ENREGISTREMENT

Pour i de 1 a n faire

Si (T[i].CA > max) *alors*

max ← T[i].CA; indmax ← i ; *finsi*

Si (T[i].CA < min) *alors*

min ← T[i].CA; indmin ← i ; *finsi*

S ← S + T[i].CA;

finpour

ecrire ("le client ayant le chiffre d'affaire le plus faible est: ");

ecrire(T[indmin].code);

ecrire(T[indmin].nom);

ecrire(T[indmin].CA);

ecrire ("le client ayant le chiffre d'affaire le plus élevé est: ");

ecrire(T[indmax].code);

ecrire(T[indmax].nom);

ecrire(T[indmax].CA);

ecrire ("la moyenne des chiffres d'affaire est: ", S/n);

fin

ENREGISTREMENT

Exercice 5:

Ecrire l'algorithme permettant d'initialiser un tableau de 50 étudiants dont chacun est caractérisé par le nom , le prénom, le numéro de la carte d'étudiant, la classe à la quelle il est affecté , cette dernière définie par la filière (exemple:LAS, LF, TMW...) et le niveau(1,2, 3,..).

Un tableau de 10 matières dont chacune est caractérisée par le nom de la matière et la note obtenue. Aussi décrit par sa moyenne et son rang.

Après initialisation du tableau, calculer la moyenne et le rang de chaque étudiant.

ordonner les étudiants selon leur niveau.

Afficher enfin le tableau des étudiants triés, pour cela vous devez définir les sous-programmes suivants:

Initialiser_tableau, Calcul_moy_etud, fournir_rang,
Trier_tab_etud.

SOLUTION: EXERCICE 5

Algorithme ex5

Type

structure Classe

Fil: chaîne de caractères

Niv: entier

fin structure classe

structure Matiere

nommat: chaîne de caractères

Note: réel

fin structure matière

structure Etudiant

Nom: chaîne de caractères

Prenom: chaîne de caractères

NCE: entier

C: classe

Moy: réel

rang: entier

tableau M[10]: matière

fin structure etudiant

SOLUTION: EXERCICE 5

Procédure initialiser_tab(var tableau E[50] :etudiant)

var i,j:entier

debut

pour i de 1 a 50 faire

Ecrire («donner le nom: »); lire (E[i].nom);

Ecrire («donner le prenom: »); lire (E[i].prenom);

Ecrire («donner le num de carte d'etudiant: »); lire (E[i].NCE);

Ecrire («donner la filière: »); lire (E[i].C.Fil);

Ecrire («donner le niveau: »); lire (E[i].C.Niv);

pour j de 1 a 10 faire

Ecrire («donner le nom de la matiere: »); lire (E[i]. M[j]. nommat);

Ecrire («donner le nom de la note: »); lire (E[i]. M[j]. Note);

finpour

finpour

Fin

Fonction Calcul_moy_etud(E:etudiant):réel

Var i:entier, S:réel

Debut

$S \leftarrow 0;$

pour ide 1 a 10 faire

$S \leftarrow S + E.M[i]. Note);$

Finpour

$Moy_etud \leftarrow S/10;$

fin

SOLUTION: EXERCICE 5

fonction rang_etud(E:etudiant, tableau T[50]:etudiant):entier

var i,R:entier

debut

R ← 1;

pour i de 1 a 50 faire

si (E.moyenne < T[i]. moy) alors R ← R+1 fin si

Finpour

Rang ← R

Fin

Procedure Trier_tab_etud (Var tableau E[50]:etudiant)

Var i,j:entier, Aux:etudiant

Debut

pour i de 1 a 50 faire

pour j de i+1 a 50 faire

si (E[j].C.niv < E[i].C.niv) alors Aux ← E[j]; E[j] ← E[i];
E[i] ← Aux;

fin si

Finpour

Finpour

fin

SOLUTION: EXERCICE 5

Procédure Affichage(tableau E[50]:etudiant)

Var i,j:entier,

Debut

pour i de 1 a 50 faire

Ecrire («nom: » ,E[i].nom);

Ecrire («prenom: » , E[i].prenom);

Ecrire («le num de carte d'etudiant: » , E[i].NCE);

Ecrire («filière: » , E[i].C.Fil);

Ecrire («le niveau: » , E[i].C.Niv);

pour j de 1 a 10 faire

Ecrire («nom matiere: » ,E[i]. M[j].nommat);

Ecrire («note matiere: » ,E[i]. M[j].note);

Finpour

Ecrire («moyenne: » ,E[i].moy);

Ecrire («rang: » ,E[i].rang);

Finpour

fin

SOLUTION: EXERCICE 5

Var

Tableau $T[50]$: *etudiant*

i: entier

Debut

initialiser_tab(T);

pour i de 1 a 50 faire

$T[i].moy \leftarrow \text{Calcul_Moy_etud}(T[i]);$

Finpour

pour j de 1 a 50 faire

$T[i].rang \leftarrow \text{rang_etud}(T[i], T);$

Finpour

Trier_tab_etud(T);

affichage(T);

Fin

LES FICHIERS SEQUENTIELS

I. Définition:

De point de vue logique un fichier est une suite d'articles (enregistrement) ou d'éléments; dans le cas où les articles sont uniquement des caractères on parle de fichier texte.

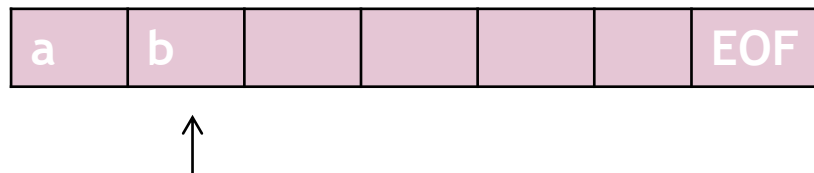
Notation:



Fin de fichier: EOF (End of File)

Fenêtre (Tampon) elle est notée F ↑ contenant la variable dont on est positionné (elle est de même type d'un élément de fichier)

Dans cet exemple la fenêtre contient b



LES FICHIERS SEQUENTIELS

II. Opération sur les fichiers:

Pour utiliser un fichier, on effectue plusieurs fonctions à savoir:

1. Ouverture d'un fichier:

L'ouverture nécessite l'indication d'un des mots suivants:

- mode lecture: l
- mode écriture: e
- mode ajout: a

La syntaxe générale de cette fonction est ouvrir(F, m)

Nom logique
du fichier

Mode
d'ouverture

Ouvrir(F,l)

F



F↑

LES FICHIERS SEQUENTIELS

Ouvrir(F,l)

Ouvrir(F,l) positionne le premier élément du fichier, s'il existe, si le fichier n'existe pas, ouvrir(F,l) provoque une erreur: Fichier inexistant

Ouvrir(F,e)

Ouvrir(F,e) permet la création d'un nouveau fichier, si F existe à l'avance il sera écrasé et F se réduit à F. EoF

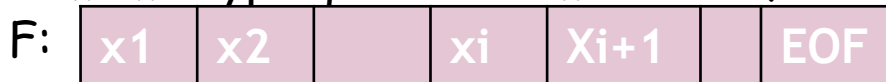
Ouvrir(F,a)

Ouvrir(F,a) c'est un mode d'écriture à la fin du fichier qui reste intacte mais avec déplacement de son EoF

2. Lecture à partir d'un fichier:

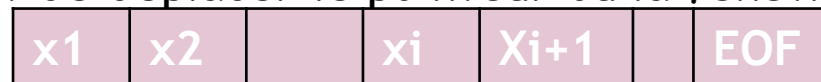
Lire(F,V)

V : variable de même type que les éléments du fichier



Lire(F,V)  (V=xi)

Lire(F,V) permet d'affecter xi (le contenu qui est pointé par la fenêtre dans le fichier) dans la variable V et de déplacer le pointeur ou la fenêtre vers l'élément suivant et on a F



LES FICHIERS SEQUENTIELS

Remarque:

On ne peut pas lire Eof

Eof(F): c'est une fonction à valeur booléenne

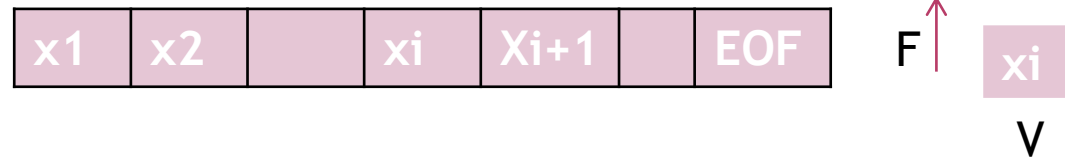
Eof(F): vrai si on est positionné sur Eof

Eof(F): Faux sinon

lire(F,V)



lire(F,V)



3. Ecriture dans un fichier:

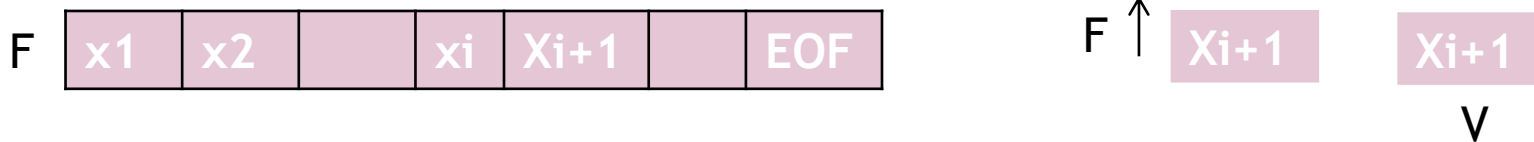
Ecrire(F,V)



Nom logique
du fichier

Variable(de même type que les éléments du fichier)

Ecrire(F,V)



LES FICHIERS SEQUENTIELS

4. Fermeture d'un fichier:

Fermer(F)  Nom logique du fichier à fermer

Exercice 1:

Soit F un fichier d'entiers, écrire l'algorithme qui permet de générer un fichier G contenant les carrés des éléments de F

Algorithme Carre

Var

F,G:fichier d'entier

V:entier

Debut

ouvrir (F,l);

ouvrir (G,e);

répéter

lire (F,V);

ecrire(G,VxV);

Jusqu'à (EoF(F)=vrai)

Fermer (F);

Fermer (G);

Fin

LES FICHIERS SEQUENTIELS

Exercice 2:

écrire l'algorithme qui permet, a partir de deux fichiers de caracteres triés de générer un fichier F3 trié resultat de la fusion de ces deux fichiers.

Algorithme fusion
Var F1,F2,F3:fichier de caracteres V,W:caractere

Debut

```
ouvrir (F1,l)
ouvrir (F2,l)
ouvrir (F3,e)
lire (F1,V); lire (F2,W);
tantque (Eof(F1)=faux) et (Eof(F2)=faux) faire
    si V<W alors        ecrire(F3,V); lire(F1,V);
    sinon                ecrire(F3,W); lire(F2,W);
finsi
Fintantque
tantque (NonEof(F1)) faire
    ecrire(F3,V);lire(F1,V);
fintantque
tantque (NonEof(F2)) faire
    ecrire(F3,W);lire(F2,W);
fintantque
Fermer(F1); Fermer(F2); Fermer(F3);
```

Fin

LES FICHIERS SEQUENTIELS

5. Assigner un fichier

Assigner(Nom physique,nom logique)

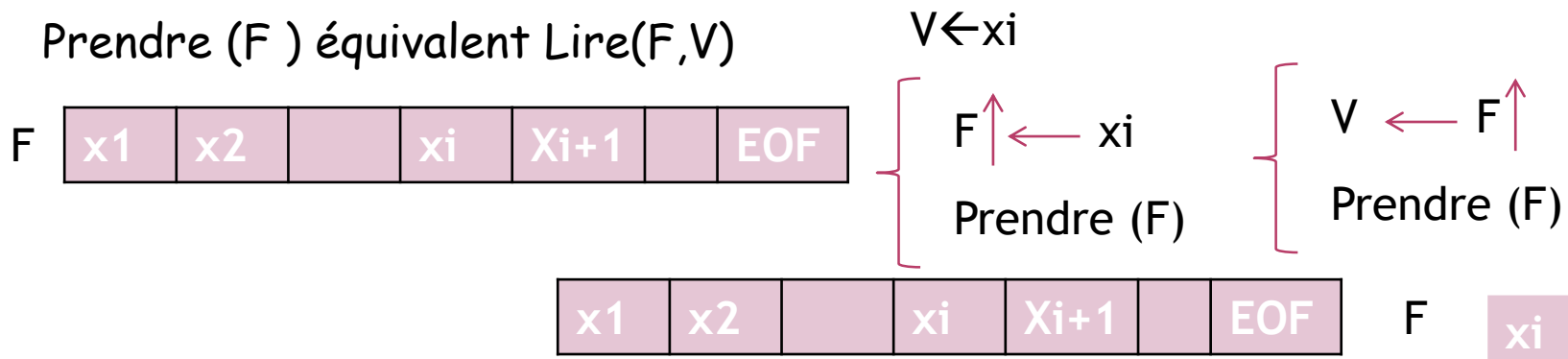
Exemple

Assigner("C:\fichier1>txt",F)

Cette fonction permet de faire la correspondance entre le fichier stocké sur disque dur (nom physique) et celui utilisé au niveau de la mémoire centrale (nom logique)

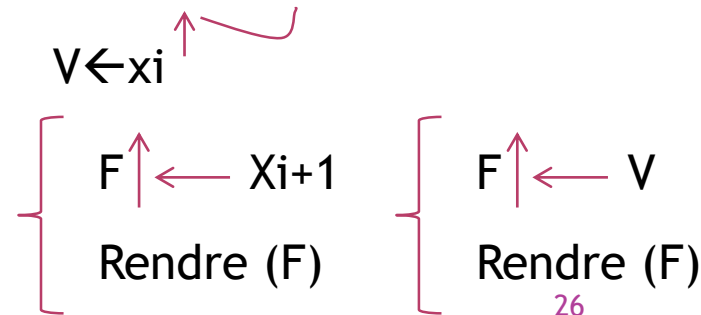
6. Prendre (nom logique fichier)

Prendre (F) équivalent Lire(F,V)



7. Rendre (nom logique fichier)

Rendre (F) équivalent écrire(F,V)



LES FICHIERS SEQUENTIELS

Exercice 3:

Reprendre l'exercice du calcul du carré des éléments d'un fichier F dans un fichier G .

Algorithme Carre2

Var

NF,NG:chaîne de caractères

F,G:fichier d'ENTIER

V:entier

Debut

Ecrire("Saisir le nom de fichier de depart"); lire (NF);

Ecrire("Saisir le nom de fichier de résultat"); lire (NG);

Assigner(NF,F);

Assigner(NG,G);

ouvrir (F,l); ouvrir (G,e)

tantque (non Eof(F)) faire

Lire (F,V);

*Ecrire (G, $v*v$);*

Fintantque

Fermer(F); Fermer(G);

Fin

LES FICHIERS SEQUENTIELS

Exercice 4:

Écrire l'algorithme permettant de calculer la somme des éléments d'un fichier F non vide d'entier.

Exercice 5:

Écrire l'algorithme permettant d'effectuer l'éclatement d'un fichier en deux, le premier contiendra les nombres pairs et le deuxième ceux impairs

Exercice 6:

Écrire l'algorithme permettant l'accès au $k^{\text{ème}}$ élément d'un fichier donné

LES FICHIERS SEQUENTIELS

Algorithme Somme

Var

NF: chaîne de caractère = « c:\fichierdesnotes.txt »

F: fichier d'entiers

V, S: entier

Debut

assigner (NF, F);

ouvrir (F, l);

S ← 0;

Repeter

lire(F, V);

S ← S + V;

jusqu'à (EOF(F))

ecrire ("la somme des elements est:", S);

fermer(F);

Fin

LES FICHIERS SEQUENTIELS

Algorithme eclatement

Var

F,F1,F2:fichier d'entiers

V:entier

Debut

ouvrir (F,l); ouvrir (F1,e); ouvrir (F2,e);

Repeter

lire(F,V);

si (V mod 2=0)alors ecrire(F1,V);

sinon ecrire(F2,V);

finsi

jusqu'à (EOF(F))

fermer(F); fermer(F1);

fermer(F2);

Fin

LES FICHIERS SEQUENTIELS

Algorithme Acces

Var

F: fichier d'entiers

V, K, i: entier

Debut

ouvrir (F, I);

ecrire ("donner la position de l'élément recherché:");

lire(K);

i ← 0;

Repeter

lire(F, V);

i ← i + 1;

jusqu'à (EOF(F)) or (i = k)

si (non EOF(F)) alors

ecrire ("l'éléments de la position:", k, "est:" , V);

sinon

ecrire ("Fin de fichier")

Fin si

fermer(F);

Fin

LES LISTES

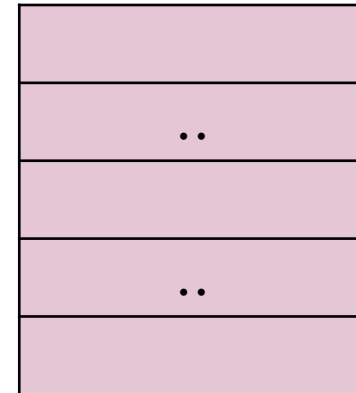
I. Les pointeurs:

Un pointeur est une variable qui contient l'adresse d'une autre variable.

Exemple:

Soit la déclaration `x:entier` celle-ci permet d'allouer un espace mémoire pour la variable `x`,

`x`



La déclaration `x ← 2` permet d'attribuer la valeur 2 à la variable `x`,
Soit `p` une variable de type pointeur qui contient l'adresse de `x`, la déclaration utilisée est de la forme:

Var

P:pointeur

Avec pointeur est un type définie par :

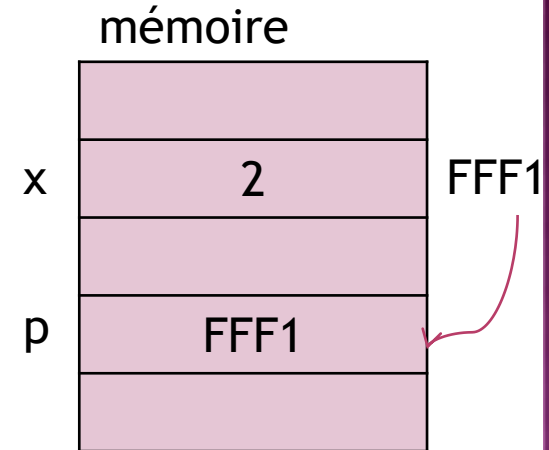
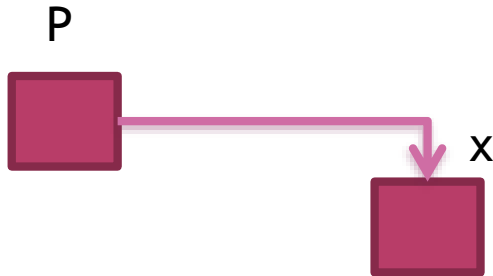
Type

Pointeur = ^objet

Ou objet est de type simple ou structuré

Soit $p : \hat{x}$ qui permet de réserver un espace mémoire pour p qui contiendra l'adresse de x , soit

Ceci est schématisée par :



Notation:

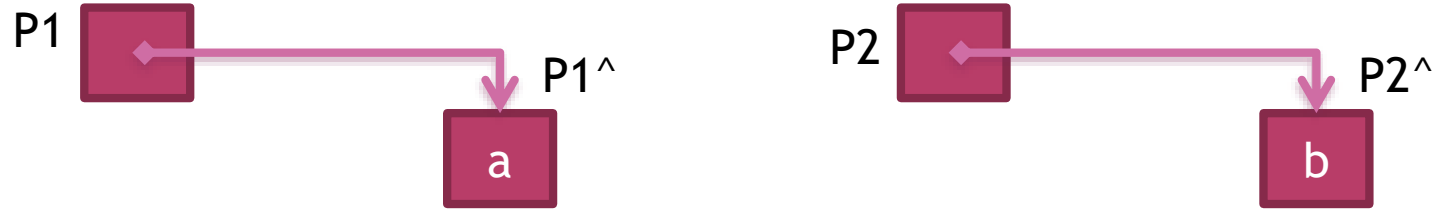
$*p^{\wedge}$ désigne l'objet dont l'adresse se trouve dans p , autrement: $p^{\wedge}=x$

Syntaxe en c:

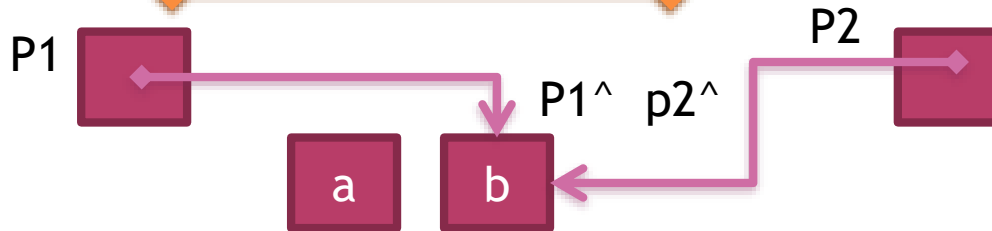
Int x; déclaration d'une variable entière
Int *p; déclaration d'un pointeur vers un entier
P=&x; mettre dans p l'adresse de x
*p contenu de l'élément pointé par p $*p=x$

exemple:

Soit $p1$ et $p2$ deux variables de type pointeur permettant respectivement l'accès à a et b :

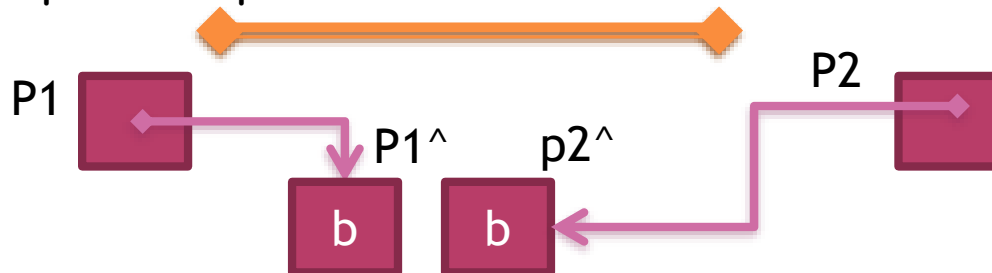


$P1 \leftarrow p2$:



On remarque que a n'est plus accessible et on obtient b par l'intermédiaire de $p1$ ou de $p2$

$P1^{\wedge} \leftarrow p2^{\wedge}$:



On remarque que a n'existe plus et b existe en deux exemplaires, l'un a une adresse dans $p1$ et l'autre a une adresse différente en $p2$

Remarque:

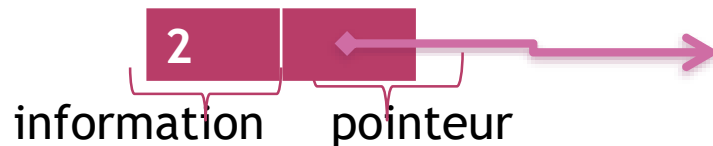
On dispose d'une variable conventionnelle de type pointeur telle que celle-ci ne correspond à aucune adresse possible d'objet » $p = \text{null}$ qui est schématisée par :



$P^{\wedge}$ n'est pas définie que si p est différent de null

II . Représentation des cellules:

Une cellule est schématisée ainsi:



Pour définir un type cellule, on utilise les structures de données on obtient ainsi:

```
Type structure Cellule
  Info:t
  P:pointeur sur cellule
Fin structure cellule
```

Ou t est un type simple, vecteur, structure

II . Représentation des listes:

a. Représentation contigue:

Type structure liste

t:tableau [lmax] d'élément

l: (1..lmax) d'élément nombre d'éléments de la liste

Fin structure liste

Var lst :liste

Lst.t[i]: acces au i^{eme} élément de la liste

b. Représentation chaînée:

Définition:

Une liste est constituée d'un ensemble de cellules chaînées entre elles. C'est l'adresse de la première cellule qui détermine la liste. Cette adresse doit se trouver dans une variable que nous appellerons très souvent **tête**.

Type structure liste

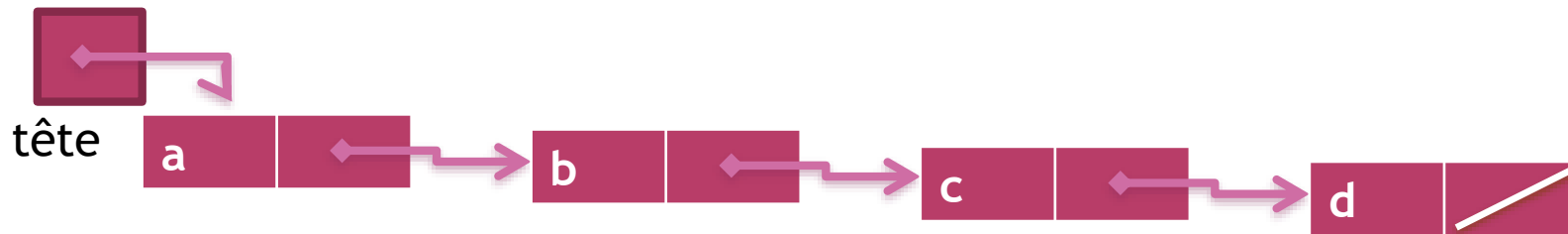
e:élément

suiv: pointeur sur liste

Fin structure liste

exemple:

Soit la liste linéaire (a,b,c,d) qui est représentée par la liste chaînée suivante:



Fonctions utilisées:

➤ **Allouer**(liste) : pointeur sur liste

Cette fonction va chercher un espace mémoire pour la liste » De ce fait, a chaque appel de cette fonction, une nouvelle cellule sera créée dont l'adresse sera retournée .

exemple:

```
Var
  p: pointeur sur liste
Debut
  P ← allouer(liste)
fin
```



Notations:

Si p est un pointeur sur liste, pour accéder à un champ de la liste on utilise la syntaxe: $p \rightarrow \text{nomchamp}$

exemple:

p:pointeur
 $P \rightarrow e \leftarrow a;$
 $P \rightarrow \text{suiv} \leftarrow \text{null}$

➤ **Liberer**(p : pointeur sur liste)

Cette fonction permet d'obtenir ou de rendre une cellule a la mémoire, elle permet de libérer la zone pour la réutiliser de nouveau.

exemple:

liberer(p)

remarques:

on parle de gestion dynamique de la mémoire, contrairement à la gestion statique des vecteurs(tableaux)

Si $p = \text{null}$, l'accès au suivant est interdit ($p \rightarrow \text{suiv}$)

IV . Algorithmes de création de liste:

(a,b)



À Compléter à partir du tableau

Insertion en queue:

Module création liste(données: nliste:entier,
resultat: tete:pointeur sur liste):

Var

p: pointeur sur liste

el:element

i:entier

dernier:pointeur sur liste

Debut

P←allouer(liste);

Lire(el)

P→e←el;

Tete←p;

Dernier←p

Pour i de 2 a nliste faire

lire (el);

p←allouer(liste);

p→e←el;

dernier→suiv←p;

Dernier←p;

Finpour

Dernier→suiv←null;

fin

Insertion en tete:

Module création liste(données: nliste:entier,
resultat: tete:pointeur sur liste):

Var

p: pointeur sur liste

el:element

i:entier

Debut

Tete←null;

Pour i de 1 a nliste faire

lire (el);

p←allouer(liste);

p→e←el;

p→suiv←tete;

tete←p;

Finpour

fin

exercices:

Exercice1: On désire généraliser ce module, en supposant que les éléments se trouvent dans un fichier F, Dans ce cas, on crée la liste à partir de son dernier élément et du 1^{er} élément du fichier

**Module création liste(données: nliste:entier,
resultat: tete:pointeur sur liste):**

```
Var
  el:element
  p:pointeur sur liste
Debut
  Ouvrir(F,l);
  Tete←null;
  Tant que (Non EOF(F)) faire
    p←allouer(liste);
    Lire(F,el);
    p→e←el;
    p→suiv←tete;
    tete←p;
  Fintantque
  Fermer(F);
fin
```

Exercice2: A partir des éléments d'un vecteur V, on crée la liste à partir de son dernier élément et du dernier élément des vecteurs aussi, on obtient alors les éléments dans le même ordre

**Module création liste(données: V:tableau[n]:element,
resultat: tete:pointeur sur liste):**

```
Var
  i:entier
  p:pointeur sur liste
Debut
  Tete←null;
  pour i de n à 1 par pas -1 faire
    p←allouer(liste);
    p→e←v[i];
    p→suiv←tete;
    tete←p;
Finpour
fin
```

Exercice3: créer la liste à partir des éléments d'un fichier en respectant le même ordre

Module création liste(données: *F:fichier d'éléments*
resultat: tete:pointeur sur liste)

```
Var
  el:element
  dernier,p:pointeur sur liste
Debut
  ouvrir(F,l);
  lire(F,el);
  p←allouer(liste);
  p→e←el;
  Tete←P;
  dernier ←P;
  tantque (notEOF(F)) faire
    p←allouer(liste);
    lire(F,el);
    p→e←el;
    dernier→suiv←p;
    dernier←p;
  Fintantque
  Dernier → suiv←null;
  Fermer(F);
fin
```

Exemples d'algorithmes sur les listes chaînées:

- ✓ Algorithmes de calcul du nombre d'élément d'une liste chaînée
- ✓ Algorithmes de calcul du nombre d'occurrence d'un élément dans une liste chaînée
- ✓ Algorithme d'accès dans une liste chaînée
- ✓ Algorithmes de mise à jour dans une liste chaînée (insertion en tête ou en queue, suppression)
- ✓ Algorithmes de recherche d'éléments dans une liste chaînée

Exercice d'application

Ecrire l'algorithme de l'insertion d'un élément au milieu de la liste c'est-à-dire à une position k donnée

Solution proposée (schéma itératif)

Procédure insert_k(var:tete: pointeur sur liste, pos:entier, el:element, var
possible:boolean)

Var

prec, p: pointeur sur liste
k:entier

Debut

possible ← faux;

si (pos=1)alors insert_tete(tete,el); possible ← vrai

sinon prec ← tete; k ← 1;

tantque (k ≠ pos-1) faire

prec ← prec → suiv;

k ← k+1;

fintantque

si (prec ≠ null) alors P ← allouer(liste);

P → e ← el;

p → suiv ← prec → suiv;

prec → suiv ← p;

possible ← vrai;

finsi

finsi

FIN

Solution proposée (schéma récursif)

Procédure insert_k(var:tete: pointeur sur liste, pos:entier, el:element, var
possible:boolean)

Var

prec, p: pointeur sur liste

k:entier

Debut

si (pos=1)alors insert_tete(tete,el); possible ←vrai;

sinon si (tete=null) alors possible ←faux;

sinon insert_k(tete → suiv,pos-1,el,possible)

finsi

finsi

FIN

LISTES DOUBLEMENT CHAINÉES

Liste doublement chaînée

Une cellule dans une liste doublement chaînée est définie par:

Structure Cellule

prec:pointeur sur cellule

info:element

suiv:pointeur sur cellule

Fin structure cellule

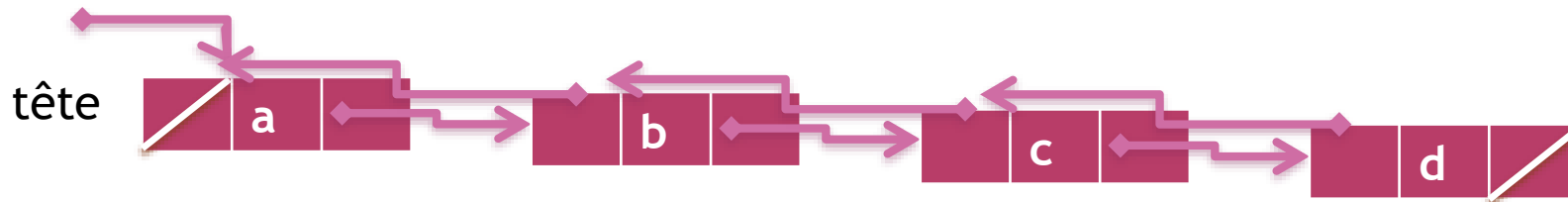
@ du précédent

info

@ du suivant

On définit alors une liste doublement chaînée

Liste:pointeur sur cellule



Insertion en tete:

Procédure *insert_tete*(var *tete*:pointeur sur liste, *el*:element):

Var

p: pointeur sur cellule

Debut

p ← allouer(cellule);

p → info ← el;

p → suiv ← tete;

tete → prec ← p;

p → prec ← null;

tete ← p;

Fin

Insertion en queue:

Procédure insert_queue(var tete:pointeur sur liste, el:element):

Var

p,pre,dernier: pointeur sur cellule

Debut

si (tete $\neq$ null) alors dernier $\leftarrow$ tete;

tantque (dernier $\neq$ null) faire

pre $\leftarrow$ dernier;

dernier $\leftarrow$ dernier $\rightarrow$ suiv;

fintantque

p $\leftarrow$ allouer(cellule);

p $\rightarrow$ info $\leftarrow$ el;

pre $\rightarrow$ suiv $\leftarrow$ p;

p $\rightarrow$ prec $\leftarrow$ pre;

p $\rightarrow$ suiv $\leftarrow$ null;

sinon

insert_tete(tete,el);

finsi

Fin

Exercice d'application

Ecrire le module d'insertion d'un élément à une position k donnée dans une liste doublement chaînée.

Procédure insert_k(var:tete: pointeur sur cellule, k:entier, el:element, var possible:boolean)

Var

der, p: pointeur sur cellule; i:entier

DEBUT

si (k=1)alors insert_tete(tete,el); possible ← vrai;

sinon i ← 1; der ← tete; possible ← faux; tant que (i < k-1) faire

der ← der → suiv ;

i ← i+1;

fintant que

si (der ≠ null)alors si (der → suiv ≠ null) alors p ← allouer(cellule);

p → info ← el;

p → prec ← der;

p → suiv ← der → suiv;

der → suiv → prec ← p;

der → suiv ← p; possible ← vrai;

sinon

insertqueue(tete,el); possible ← vrai;

finsi

finsi

finsi

FIN#

Suppression à une position k donnée

Ecrire le module la suppression d'un élément à une position k donnée dans une liste doublement.

Procédure suppression_k(var:tete: pointeur sur cellule, k:entier, el:element,
var possible:boolean)

Var
der, pk: pointeur sur cellule; i:entier

DEBUT

si (k=1)alors suppression_tete(tete,possible);

sinon i ← 1; der ← tete; possible ← faux; tant que (i < k-1) faire
der ← der → suiv ;
i ← i+1;

fintantque

si (der ≠ null)alors pk ← der → suiv;

si (pk → suiv ≠ null) alors der → suiv ← pk → suiv;
pk → suiv → prec ← der;
liberer(pk);
possible ← vrai;

sinon

suppressionqueue(tete,possible);

finsi

finsi

finsi

FIN

LISTES CIRCULAIRES

Listes Circulaires

On a une liste L (a,b,c,d) dont le suivant du dernier élément d est le premier élément a:

